

Programas Descartáveis sob Especificação Imutável: o que um Validator Realmente Garante — um Estudo com Extração de Dados

Carlos Eduardo Dias Batista
Brasil

Abstract

Data extraction pipelines break when sources change format. Intent-Driven Adaptive Extraction (IDAE) inverts the usual relation between program and intent: the extraction program is a disposable artifact synthesized on demand by an LLM, while an immutable final intention (the *telos*), checked by a validator, is the permanent artifact. Synthesized programs are amortized over all records of the same format, and a two-phase intervention hierarchy first re-synthesizes programs and then reconstructs intermediate validators. We evaluate IDAE with a methodology that separates *acceptance by the validator* from *correctness*: every record carries a ground-truth oracle, and all approaches receive the same specification and are measured by the same oracle. Seven experiments cover two synthetic domains, 560 real USGS earthquake records, two models (Claude Opus 4.5 and Claude Haiku 4.5) and six repetitions per domain. With Opus 4.5, IDAE delivered 560/560 correct USGS records with 2 LLM calls, while per-record LLM transformation reached 62.7% with 560 calls (timestamp arithmetic errors), a $\sim 120\times$ token reduction; in the main synthetic comparison, IDAE made no value errors, whereas per-record LLM output had 0.7–4.0% silent corruption. However, the oracle refutes the claim that zero hallucination is architectural: with Haiku 4.5, a single faulty program amortized over its format corrupted 98% of a phase; the reconstruction of intermediate validators (Phase 2) caused 74% silent corruption in the phases where it fired; a *telos* that required a field absent from the source made synthesized programs fabricate it in 330 records and clamp valid negative depths to zero. We also identify format poisoning: because IDAE cannot distinguish a failing program from an invalid record, one invalid record exhausts the TTL and causes valid records of the same format to be rejected. Amortizing explicit rejections fixes this without extra cost. IDAE’s guarantee is exactly the strength of its specification: it buys cost, determinism and auditability, not semantic correctness.

Keywords

Intent-Driven Programming, Program Synthesis, Data Extraction, LLM, Silent Data Corruption, Test Oracle, Empirical Evaluation

Resumo

Pipelines de extração de dados quebram quando as fontes mudam de formato. O IDAE (*Intent-Driven Adaptive Extraction*) inverte a relação entre programa e intenção: o programa de extração é um artefato descartável sintetizado por um LLM, e a intenção final imutável (o *telos*), verificada por um validator, é o artefato permanente. Avaliamos o IDAE com um oráculo de verdade-terreno por registro, aplicado igualmente a todas as abordagens, em sete experimentos com dois domínios sintéticos, 560 eventos sísmicos

reais do USGS e dois modelos (Claude Opus 4.5 e Haiku 4.5). Com o Opus 4.5, o IDAE converteu corretamente os 560 eventos com 2 chamadas ao LLM, contra 62,7% com 560 chamadas da transformação por registro ($\sim 120\times$ menos tokens). O oráculo, porém, refuta a garantia de “zero alucinação” como propriedade arquitetural: um programa errado amortizado corrompeu 98% de uma fase; a reconstrução de validators intermediários corrompeu 74% das fases em que atuou; um *telos* insatisfatório levou os programas a fabricar valores e a alterar dados reais; e um único registro inválido pode envenenar um formato inteiro. A garantia do IDAE é exatamente a força da sua especificação: ele compra custo, determinismo e auditabilidade, não correção semântica.

Palavras-chave: programação orientada a intenção, síntese de programas, extração de dados, LLM, corrupção silenciosa de dados, oráculo de teste.

DOI: <https://doi.org/10.5281/zenodo.23050791>

1 Introdução

Pipelines de extração e transformação de dados codificam o *processo* de transformação: o programa é o artefato permanente e a intenção — o que o dado final deveria satisfazer — permanece informal, dispersa em documentação e na memória da equipe. Quando uma API muda seu schema ou uma fonte passa a exportar outro formato, alguém precisa reescrever o parser [20]. A indústria discute uma transição de desenvolvimento “code-first” para “intent-first” [23], e a ideia de elevar o nível de abstração tem raízes na programação declarativa [16] e na síntese de programas [9].

O IDAE (*Intent-Driven Adaptive Extraction*) operacionaliza essa inversão para extração de dados: o programa é descartável, sintetizado por um LLM a partir da intenção, e o *telos* — a intenção final, verificada por um validator — é imutável. Programas aprovados são reutilizados (amortizados) para todos os registros do mesmo formato; quando o formato muda, o programa é descartado e outro é sintetizado.

A promessa mais forte dessa arquitetura é semântica: se todo dado aceito passa pelo validator do *telos*, o sistema não aceitaria “alucinações”. Essa promessa depende de uma distinção que costuma ser negligenciada: **ser aceito pelo validator não é o mesmo que estar correto**. Um validator verifica formato e restrições declaradas; ele não conhece o valor verdadeiro de cada campo — é o clássico problema do oráculo em teste de software [5]. Uma avaliação que mede “alucinação” como “falha no validator”, ou que só mede alucinação para os baselines, confirma a arquitetura por construção.

Este artigo avalia o IDAE com um oráculo de verdade-terreno por registro, aplicado igualmente a todas as abordagens, e responde seis questões de pesquisa:

- **RQ1** (correção): os mecanismos do IDAE — cadeia de intenções, compressão, Fase 2 — entregam o telos *correto*, e não apenas aceito?
- **RQ2** (custo): quanto a amortização de programas reduz chamadas e tokens em relação à transformação por registro, a correção comparável?
- **RQ3** (garantia semântica): o validator impede corrupção silenciosa? De que ela depende?
- **RQ4** (entradas adversariais e deriva): como o IDAE se comporta quando parte dos registros deve ser rejeitada?
- **RQ5** (componentes): qual a contribuição de cada mecanismo e de variantes que corrigem as falhas observadas?
- **RQ6** (convergência): o Índice de Resolução H_r e o TTL descrevem a convergência semântica do Negotiator?

1.1 Contribuições

- (1) **Framework IDAE**: arquitetura com telos imutável, programas descartáveis amortizados por formato, hierarquia de intervenção em duas fases e compressão de cadeia (Seções 2–3).
- (2) **Metodologia de avaliação com oráculo**: separação operacional entre aceitação e correção (*corrupção silenciosa*), mesma especificação para todas as abordagens, testes estatísticos exatos e registro auditável de todas as chamadas (Seção 4).
- (3) **Evidência de custo**: em 560 registros sísmicos reais, 100% de correção com 2 chamadas contra 62,7% com 560 chamadas da transformação por registro (~120× menos tokens).
- (4) **Refutação da garantia arquitetural**: a garantia do IDAE é exatamente a força da sua especificação. Documentamos quatro mecanismos de corrupção silenciosa — amortização de um programa errado, relaxamento de validators na Fase 2, fabricação induzida por telos mal-especificado e “validator hacking” pelo programa sintetizado — e um mecanismo de rejeição falsa, o *envenenamento de formato*.
- (5) **Correções de projeto**: rejeição explícita amortizada (elimina o envenenamento sem custo adicional) e validação ancorada com N-version [4], avaliadas por ablação e repetições.

2 Fundamentação

2.1 Intenção como especificação executável

Dado um telos ι com validator V_ι e uma entrada x , o IDAE busca um programa σ tal que $V_\iota(\sigma(x)) = \text{True}$. O programa σ é descartável; ι é permanente [3, 15]. Diferentemente da síntese de programas tradicional [9], o produto não é o programa, e sim o dado certificado: σ é descartado quando deixa de satisfazer V_ι .

2.2 Aceitação, correção e corrupção silenciosa

Seja $y^*(x)$ a saída verdadeira de x — ou $\perp$, se x deve ser rejeitado. Uma abordagem decide $\hat{y}(x)$ (aceita com saída) ou rejeita. Definimos, por registro:

- **correto**: aceito e $\hat{y}(x) = y^*(x)$ (com tolerância numérica);

- **corrupção silenciosa**: aceito e $y^*(x) = \perp$ ou $\hat{y}(x) \neq y^*(x)$;
- **rejeição correta**: rejeitado e $y^*(x) = \perp$;
- **rejeição falsa**: rejeitado e $y^*(x) \neq \perp$.

A “alucinação relativa ao validator” — saída aceita que viola V_ι — é zero por definição em qualquer sistema que filtre por V_ι . A corrupção silenciosa é a quantidade relevante: dados errados que o sistema certifica como válidos.

2.3 Índice de Resolução (H_r)

O Índice de Resolução mede a fração de campos que ainda não atingem o objetivo após uma tentativa do Negotiator: $H_r = |\{f \in F : f \text{ incorreto}\}|/|F|$. Medido pelo validator, “incorreto” significa “reprovado”; aqui medimos também H_r contra o oráculo, o que permite observar programas aprovados pelo validator com $H_r > 0$.

3 Arquitetura do IDAE

Tabela 1: Componentes do IDAE e sua mutabilidade

Componente	Responsabilidade	Mutável?
IntentionStore	Intenções semânticas e encadeamento	Nunca
ValidatorStore	Validators; telos final imutável	Intermediários apenas
ProgramStore	Programa ativo por intenção/formato	Descartado
Negotiator	Hierarquia de intervenção em 2 fases	Fixo
Synthesizer	Síntese via LLM; reconstrói validators	Na quebra
CompressionEngine	Suspensão/reativação de intenções	Fixo

Hierarquia de intervenção. Quando o programa vigente produz uma saída reprovada: (1) o programa é descartado; (2) *Fase 1*: o Negotiator sintetiza novos programas (TTL = 3); (3) *Fase 2*: se esgotada e a intenção não for o telos, o validator intermediário é reconstruído pelo LLM usando o telos como âncora e um novo programa é sintetizado (TTL = 2); (4) caso contrário, lança-se `SystemicInconsistencyError`. O telos nunca é alterado. O LLM recebe apenas a intenção e a amostra que falhou, nunca o programa anterior.

Amortização. O programa aprovado é reutilizado para os registros seguintes do mesmo formato, sem chamar o LLM. O LLM sai do caminho crítico: o custo depende do número k de formatos distintos, e não do número N de registros. Se um formato esgota o TTL, é marcado como esgotado e seus registros seguintes são rejeitados sem nova síntese.

Compressão. Após a intenção 1, o CompressionEngine testa a entrada contra os validators das intenções intermediárias; se ela já satisfaz V_k , as intenções até k são suspensas para aquele fluxo e reativadas quando os dados mudam. O campo `chain_path` registra o caminho percorrido.

4 Metodologia de Avaliação

4.1 Oráculo e domínios

Todos os registros sintéticos são gerados com sua verdade-terreno por um PRNG com semente. Cada domínio tem seis fases: cinco de variação de formato e uma adversarial com dez variantes (sete que *devem* ser rejeitadas e três aceitáveis, porém ardilosas).

Financeiro (telos: `audit_id, value_in_usd, date DD/MM/AAAA` em UTC, category, status): JSON limpo; JSON renomeado com

offset −03:00 (20% dos registros viram o dia em UTC); pré-agregado com valor já em USD; pipe com moeda por extenso, número no formato brasileiro e mês em português; CSV sem cabeçalho com colunas reordenadas. As variantes adversariais incluem moeda ausente, valor zero, negativo ou não numérico, data ausente, moeda desconhecida e registro de outro domínio.

IoT (telos: `device_id`, `metric`, `value` na unidade canônica, `timestamp_utc`, `status` derivado de limiares, `zone`): JSON limpo; fornecedor com campos e zonas codificados; Fahrenheit; pipe legado (F, %, mbar); CSV em Kelvin. As variantes adversariais incluem métrica, timestamp ou zona ausentes, valor "NaN", valores fisicamente impossíveis e registro financeiro.

USGS: 560 eventos reais do USGS Earthquake Catalog (GeoJSON M4+, CSV de redes mistas, GeoJSON de micro-eventos, CSV bruto e fluxo misto intercalado). O oráculo é derivado deterministicamente dos campos-fonte. Avaliamos dois telos: o *estricto*, que exige significance inteiro e profundidade ≥ 0 , e um *corrigido*, com significance nulo quando ausente na fonte (o CSV do USGS não tem o campo `sig`) e profundidade ≥ -10 km (o USGS registra profundidades negativas, acima do datum).

4.2 Abordagens

Todas recebem o mesmo texto de especificação, incluindo a regra de rejeição (“se o registro não pode ser convertido fielmente, rejeite; nunca adivinhe”):

- **Schema-Based**: parser determinístico escrito à mão para o formato da Fase 1.
- **LLM-Direct**: uma chamada por registro; o LLM devolve o JSON final ou null; sem validator.
- **LLM+Val+Retry**: como o LLM-Direct, mas com o validator do telos como portão e até 3 tentativas; sem amortização.
- **IDAE**: síntese com TTL = 3, validator como portão, amortização, marcação de formato esgotado, com programas indexados por assinatura de formato.
- **IDAE-cadeia**: porte fiel do experimento de cadeia (4 intenções, compressão, Fase 2), incluindo seus prompts.
- **Variantes** (E4, E5): sem validator; sem amortização; *validator ancorado* (o identificador deve aparecer literalmente na entrada e, no IoT, o status deve ser coerente com o valor); *rejeição explícita amortizada* (null do programa rejeita o registro sem descartar o programa, e um programa que rejeita é mantido para o formato); *N-version* com $k = 2$ (dois programas independentes devem concordar); e *IDAE+*, que combina as três.

4.3 Modelos e protocolo

O modelo principal é o **Claude Opus 4.5** (identificador `claude-opus-4.5`). O **Claude Haiku 4.5** é usado como modelo menor na análise de sensibilidade. Os programas sintetizados executam em sandbox (node: vm, timeout de 1 s) com fuso horário fixo (America/Sao_Paulo), de modo que o uso de datas locais em vez de UTC produz erro observável. Todo prompt e toda resposta são registrados em JSONL. Tokens são estimados como caracteres/4.

No E2, as abordagens por registro rodam numa subamostra estável (20 registros por fase normal e toda a fase adversarial, $n =$

150 por domínio); o IDAE e o Schema-Based rodam no fluxo completo (100 por fase e 50 adversariais, $n = 550$) e também são reportados na subamostra, pareados registro a registro. Comparações pareadas usam McNemar exato; comparações entre execuções usam Wilcoxon signed-rank exato e o δ de Cliff [2]; proporções têm IC de Wilson a 95%.

5 Resultados

5.1 E1 — Cadeia de intenções, compressão e Fase 2 (RQ1)

A cadeia de 4 intenções processou 1.000 registros em 5 fases (200 por fase) com 56 chamadas ao LLM. A compressão funcionou como projetado: os caminhos [1, 2, 3, 4] (595 registros), [1, 3, 4] (200, dados pré-enriquecidos) e [1, 4] (200, pré-agregados) foram descobertos sem intervenção. Apenas 5 registros (0,5%) foram rejeitados. **Porém, 297 dos 995 registros aceitos (29,8%) estavam errados** (Tabela 2).

Tabela 2: E1 — Cadeia de 4 intenções (Opus 4.5, $n = 200$ por fase)

Fase	Caminho	Correto	Corrupção	Campos errados
P1 JSON limpo	[1,2,3,4]	200	0	—
P2 pré-enriquecido	[1,3,4]	200	0	—
P3 pipe, moeda por extenso [†]	[1,2,3,4]	50	150	valor (150), categoria (50)
P4 pré-agregado	[1,4]	200	0	—
P5 CSV informal [†]	[1,2,3,4]	48	147	valor (147), categoria (49)

[†]Fases em que a Fase 2 reconstruiu o validator da intenção 1.

As corrupções concentram-se **exatamente nas fases em que a Fase 2 atuou**. O validator original da intenção 1 exige código ISO de moeda; diante de “Reais” ou “libra”, a Fase 1 esgotou e a Fase 2 reconstruiu um validator mais flexível, que passou a aceitar nomes por extenso. O programa da intenção 2, sintetizado na Fase 1 para códigos ISO e ainda válido segundo seu próprio validator, não reconheceu os nomes e aplicou taxa 1: só os registros em dólar (25%) saíram corretos, e registros em reais foram classificados como internacionais. O telos imutável não detectou nada, porque verifica apenas o formato. **O relaxamento de um validator intermediário propaga-se silenciosamente pela cadeia**: a Fase 2, apresentada como mecanismo de recuperação, é um mecanismo de corrupção quando os programas a jusante não são revalidados contra a nova semântica. Nas fases sem Fase 2, a correção foi de 100%.

5.2 E2 — Comparação com baselines sob oráculo (RQ2–RQ4)

Com o Opus 4.5, o IDAE não cometeu nenhum erro de valor em 1.100 registros e usou 11–14× menos tokens por registro que a transformação por registro — diferença que cresce com N , pois o custo do IDAE depende de k e não de N . As corrupções do LLM-Direct são do tipo que um validator de formato não detecta: aritmética errada na conversão Fahrenheit→Celsius (−4,54 em vez de −3,99) e a linha pipe inteira copiada como identificador. O LLM+Val+Retry comete os mesmos erros, o que confirma que o validator de formato não os filtra; o programa sintetizado, por fazer a aritmética em código, não erra.

Tabela 3: E2 — Opus 4.5. IDAE e Schema-Based no fluxo completo ($n = 550$); LLM-Direct e LLM+Val+Retry na subamostra ($n = 150$). Máximo de “Correto” possível: 90,9% (fluxo) e 76,7% (subamostra).

Dom.	Abordagem	Correto	Corrupção	Rej. falsa	Chamadas	Tokens/reg.
Fin.	Schema-Based	19,1%	0,0%	410	0	0
	IDAE	90,9%	0,0%	15	17	27
	LLM-Direct	72,7%	4,0%	0	150	387
	LLM+Val+Retry	74,0%	2,7%	0	150	387
IoT	Schema-Based	19,1%	0,0%	410	0	0
	IDAE	90,9%	0,0%	15	20	43
	LLM-Direct	76,0%	0,7%	0	150	497
	LLM+Val+Retry	76,0%	0,7%	0	150	497

Entradas inválidas foram rejeitadas por todos. Com a mesma especificação e a regra de rejeição explícita, as três abordagens baseadas em LLM rejeitaram corretamente as 35 entradas inválidas de cada domínio — inclusive o LLM-Direct, sem validador. Com a regra de rejeição na especificação, rejeitar entradas inválidas não dependeu do validador.

O IDAE rejeitou falsamente as 15 entradas adversariais aceitáveis (moeda minúscula com valor em string, número brasileiro, virada de dia em UTC; pressão em kPa, vírgula decimal, limite crítico). Elas compartilham a assinatura de formato de registros inválidos que as precederam. Como o IDAE não distingue “o programa falhou” de “o registro é inválido”, um registro inválido faz o programa vigente ser descartado, a Fase 1 esgota o TTL tentando satisfazer o validador com um registro que não deve ser satisfeito, e o formato é marcado como esgotado: os registros válidos seguintes daquele formato são rejeitados sem nova síntese. Chamamos esse efeito de **envenenamento de formato**. Na subamostra pareada, o LLM-Direct tomou mais decisões corretas que o IDAE (McNemar exato: 15 vs 6, $p = 0,078$ no financeiro; 15 vs 1, $p < 0,001$ no IoT), inteiramente por causa desse efeito.

Sensibilidade ao modelo (Haiku 4.5). Com o modelo menor (Tabela 4), o IDAE teve **21,5% de corrupção silenciosa** no financeiro. Dois programas explicam tudo: na Fase 5, o programa sintetizado leu o valor com a escala errada (3.816,58 BRL $\rightarrow$ 76.331,60 USD), e o validador, que exige apenas valor positivo, aprovou; a amortização aplicou o erro a 98 dos 100 registros da fase. Na Fase 2, o programa usou a data local em vez da UTC, errando os 20 registros noturnos. O IoT não teve corrupção, mas duas fases inteiras foram rejeitadas (115 rejeições falsas). A transformação por registro com Haiku erra de forma dispersa (~ 15 – 21%); o IDAE erra de forma **concentrada e sistemática**: 0% ou 98% por fase, conforme a qualidade de um único programa.

Tabela 4: E2 — Haiku 4.5 (mesmos fluxos e subamostras)

Dom.	Abordagem	Correto	Corrupção	Rej. falsa	Chamadas
Fin.	IDAE	51,3%	21,5%	115	19
	LLM-Direct	59,3%	20,7%	0	150
	LLM+Val+Retry	59,3%	17,3%	0	160
IoT	IDAE	72,7%	0,0%	115	22
	LLM-Direct	62,0%	21,3%	0	150
	LLM+Val+Retry	62,0%	14,7%	0	170

O validador do telos reduziu a corrupção por registro (LLM-Direct $\rightarrow$ LLM+Val+Retry: de 20,7% para 17,3% e de 21,3% para 14,7%), eliminando os registros que deveriam ter sido rejeitados. Os erros de valor, de data e de status coerente com o valor, porém, passaram.

5.3 E3 — Dados reais do USGS (RQ2, RQ3)

Tabela 5: E3 — 560 eventos reais do USGS (Opus 4.5)

Telos	Abordagem	Correto	Corrupção	Chamadas	Tokens
Corrigido	IDAE	100%	0%	2	$\sim 2,7k$
	IDAE+	100%	0%	4	$\sim 5,6k$
	LLM-Direct	62,7%	37,3%	560	$\sim 325k$
Estrito	IDAE	40,4%	59,6%	5	$\sim 6,8k$
	LLM-Direct	2,0%	98,0%	560	$\sim 295k$

Com o telos corrigido, o IDAE converteu corretamente **todos os 560 eventos reais com 2 chamadas** (um programa para GeoJSON e outro para CSV, incluindo aspas com vírgulas no campo place), contra 62,7% do LLM-Direct com 560 chamadas — redução de $\sim 120\times$ em tokens. Os 208 erros do LLM-Direct são de timestamp: o modelo converteu “de cabeça” o tempo em milissegundos desde a época para ISO 8601 e errou horas e dias. É o mesmo padrão do E2: aritmética feita pelo LLM erra, aritmética feita pelo programa sintetizado não erra.

O telos estrito induz fabricação. Ele exige significance inteiro, campo inexistente no CSV do USGS. O IDAE aceitou 330 registros CSV com o valor inventado (327 com significance = 0) e aceitou os 11 eventos com profundidade negativa (7 deles entre os 330 do CSV, totalizando 334 corrupções) depois que o programa sintetizado **substituiu a profundidade por 0** para satisfazer a restrição ≥ 0 . Esse é um caso de *validator hacking* pelo próprio programa: quando a especificação é insatisfatória para um dado real, o sintetizador encontra um programa que a satisfaz alterando o dado.

5.4 E4 — Ablação e variantes (RQ5)

Cada variante remove ou adiciona um único mecanismo sobre o mesmo fluxo do E2 (Opus 4.5, mesma semente). A Fase 2 não se aplica aqui (intenção única e imutável); seu efeito foi medido no E1.

Tabela 6: E4 — Ablação ($n = 550$ por domínio; “sem amortização” na subamostra de 150, cujo máximo de “Correto” é 76,7%)

Variante	Financeiro			IoT		
	Corr./Corrup.	Rej.f.	Cham.	Corr./Corrup.	Rej.f.	Cham.
IDAE (original)	90,9 / 0,9	15	16	90,9 / 0,0	15	20
sem validator	90,9 / 0,9	15	17	90,9 / 0,0	15	20
validator ancorado	90,9 / 0,0	15	17	90,9 / 0,0	15	20
rejeição explícita	92,7 / 0,9	0	8	93,6 / 0,0	0	9
N-version ($k = 2$)	90,9 / 0,0	15	22	90,9 / 0,0	15	25
IDAE+ (todas)	92,7 / 0,0	5	25	93,6 / 0,0	0	30
sem amortização	76,7 / 0,0	0	220	76,7 / 0,0	0	220

O validador de formato não fez diferença mensurável. “IDAE (original)” e “sem validator” produziram decisões idênticas

nos dois domínios. Nos 5 registros sem moeda do financeiro, ambos adotaram um programa que atribuiu uma moeda padrão, e o validator o aprovou. Com o Opus 4.5, os programas sintetizados raramente produzem saídas malformadas; quando erram, erram com formato válido. As variantes “ancorado” e “N-version” rejeitaram esses 5 registros, mas os registros de motivo mostram que foi por esgotamento de TTL (os programas amostrados rejeitaram), e não por ação do mecanismo: não atribuímos a elas esse efeito.

Sem amortização, a correção é máxima (76,7% de 76,7% possíveis na subamostra), sem rejeições falsas, mas com 220 chamadas para 150 registros, contra 16–20 para 550 — a amortização é a fonte do ganho de custo e também do envenenamento de formato.

Rejeição explícita amortizada eliminou as 15 rejeições falsas nos dois domínios e reduziu as chamadas pela metade (8–9 contra 16–20), porque um registro inválido deixa de custar 3 sínteses. Ela também expôs um novo erro: no financeiro, os registros com valor em string ("3440.04"), antes rejeitados por envenenamento, passaram a ser processados pelo programa amortizado da Fase 1, que interpretou o ponto como separador de milhar brasileiro (344.004 USD) — 5 corrupções aprovadas pelo validator.

N-version detectou esse erro. No IDAE+, o segundo programa independente discordou do primeiro nesses 5 registros, que foram rejeitados (nversion-disagree) em vez de corrompidos: o erro virou falha segura. O custo foi de 25–30 chamadas, ainda uma ordem de grandeza abaixo da transformação por registro.

5.5 E5 — Repetições independentes (RQ5)

O E5 repete, com 6 sementes por domínio (50 registros por fase e 30 adversariais; dados e amostragem do LLM diferentes a cada execução), o IDAE original, o IDAE+ e o IDAE sem validator no fluxo completo, e o LLM-Direct na fase adversarial.

Tabela 7: E5 — 6 execuções por domínio (média $\pm$ desvio-padrão, % de $n = 280$)

Dom.	Abordagem	Correto	Corrupção	Rej. falsa	Chamadas
Fin.	IDAE (original)	89,3 $\pm$ 0,0	0,2 $\pm$ 0,4	3,2 $\pm$ 0,0	16,8 $\pm$ 1,0
	IDAE sem validator	89,3 $\pm$ 0,0	0,2 $\pm$ 0,4	3,2 $\pm$ 0,0	16,7 $\pm$ 0,8
	IDAE+	92,5 $\pm$ 0,0	0,0 $\pm$ 0,0	0,0 $\pm$ 0,0	20,5 $\pm$ 1,0
IoT	IDAE (original)	89,3 $\pm$ 0,0	0,0 $\pm$ 0,0	3,2 $\pm$ 0,0	20,0 $\pm$ 0,0
	IDAE sem validator	89,3 $\pm$ 0,0	0,0 $\pm$ 0,0	3,2 $\pm$ 0,0	20,0 $\pm$ 0,0
	IDAE+	92,5 $\pm$ 0,0	0,0 $\pm$ 0,0	0,0 $\pm$ 0,0	22,0 $\pm$ 0,0

O IDAE+ superou o IDAE original em correção e em rejeições falsas em **todas as 6 execuções dos dois domínios** (Wilcoxon exato, $p = 0,031$, o menor valor possível com $n = 6$; δ de Cliff = 1,0), ao custo de 2–4 chamadas a mais por execução ($p = 0,031$). O IDAE original teve corrupção em 1 das 6 execuções financeiras (3 registros sem moeda aceitos com moeda inventada). O LLM-Direct, na fase adversarial, teve corrupção em 1 das 6 execuções do IoT (1 registro) e em nenhuma do financeiro. As diferenças de corrupção entre abordagens não são significativas ($p = 1,0$): com o Opus 4.5 e a regra de rejeição na especificação, a corrupção é um evento raro para todas as abordagens nesses domínios. **A remoção do validator de formato não alterou nenhuma decisão em nenhuma das 12 execuções.**

5.6 E6 — H_r e sensibilidade de TTL (RQ6)

Registramos, a cada tentativa de síntese em E4 e E5 (exceto a variante sem amortização, executada em paralelo), se o programa foi aprovado pelo validator e o H_r medido contra o oráculo no registro que disparou a síntese.

Com o Opus 4.5, todas as 327 sínteses disparadas por registros aceitáveis convergiram na primeira tentativa, com $H_r = 0$ contra o oráculo. Nenhum programa aprovado pelo validator tinha campo errado no registro de disparo, e as tentativas 2 e 3 nunca foram usadas por registros válidos. O TTL = 3 foi consumido integralmente por registros que *deveriam* ser rejeitados: 315 seqüências de síntese, que custaram 598 chamadas (65% de todas as chamadas de síntese), nas quais o validator acabou aprovando um programa que fabricava a saída em 7 casos (2,2%). **Com um modelo capaz, TTL > 1 não ajudou nenhum registro válido e apenas aumentou a exposição à fabricação** — cada tentativa extra é mais uma amostra do LLM tentando satisfazer um registro que não deve ser satisfeito. Os erros de programa observados no E2-Haiku (escala do valor, data local) também tinham $H_r > 0$ apesar da aprovação pelo validator: H_r medido pelo validator mede convergência de formato, não semântica.

5.7 E7 — Deriva gradual e telos mal-especificado (RQ3, RQ4)

Deriva. Em registros no formato da Fase 1, uma fração crescente (5, 20, 50 e 80%) chega sem moeda, e a decisão correta é rejeitá-los. Cada nível foi executado 3 vezes para o IDAE (12 execuções por variante). O **IDAE original fabricou a moeda em 2 das 12 execuções**: um programa com `data.currency || 'USD'` passou no validator e foi amortizado para todos os registros sem moeda daquele fluxo (25 e 4 registros corrompidos; 100% dos inválidos da execução). Nas outras 10, rejeitou corretamente, gastando 3 chamadas para esgotar o formato. A variante com **rejeição explícita amortizada** rejeitou corretamente em 12 de 12 execuções, com 2 chamadas em vez de 4, porque o programa que rejeita é mantido e o LLM não é reamostrado a cada registro inválido. O LLM-Direct acertou tudo com 100 chamadas por nível.

Telos mal-especificado. Com uma especificação que não exige valor positivo e um validator que não verifica o sinal, o IDAE aceitou, com uma chamada, todos os registros negativos e zerados (20 de 30; 66,7% de corrupção), com Opus e com Haiku. A “alucinação relativa ao validator” foi zero. Com a especificação correta, os inválidos foram rejeitados, mas o primeiro registro negativo envenenou o formato e **9 dos 10 registros válidos seguintes foram rejeitados**. Em execuções exploratórias, o Opus 4.5 rejeitou valores negativos mesmo sem a regra na especificação, e deixou de rejeitar moeda ausente quando a regra de rejeição foi removida: **a regra de rejeição na especificação é o que impede a fabricação**, e não o validator.

6 Discussão

O que a amortização compra. Com um modelo capaz, sintetizar um programa por formato e executá-lo deterministicamente é mais barato (~11–120× menos tokens), mais auditável (um programa inspecionável por formato, em vez de N respostas) e, neste estudo,

mais correto que pedir ao LLM que transforme cada registro: programas não erram aritmética de fuso, época ou unidade da forma como o LLM erra ao gerar texto. Esse é o resultado positivo mais robusto do IDAE.

O que a amortização custa. A mesma propriedade que torna o IDAE barato torna seus erros sistemáticos: um programa errado aprovado pelo validator é aplicado a todo o formato (98 de 100 registros no E2-Haiku; 100% dos registros sem moeda no E7). A transformação por registro erra de forma dispersa; o IDAE erra por formato. Para um consumidor downstream, erros sistemáticos são mais danosos (viés) e também mais detectáveis (um único programa a inspecionar).

A garantia é da especificação, não da arquitetura. O telos imutável garante exatamente o que o validator verifica. Todos os mecanismos de corrupção observados — amortização de programa errado, relaxamento na Fase 2, telos insatisfatório que induz fabricação e *clamping* — produziram saídas que satisfazem o validator. Afirmarções como “0% de alucinação” só são defensáveis como “0% de violação do validator”, o que é trivial.

A Fase 2 exige revalidação a jusante. Reconstruir um validator intermediário muda o contrato entre intenções. Um projeto seguro deve, no mínimo, invalidar os programas a jusante quando um validator a montante é relaxado, ou restringir a Fase 2 a relaxamentos cuja semântica o telos consiga verificar.

Rejeição é um resultado, não uma falha. O envenenamento de formato decorre de tratar “o programa devolveu null” como falha do programa. Permitir que o programa rejeite o registro, e amortizar essa rejeição, elimina as rejeições falsas e reduz chamadas (E4, E5, E7).

Implicações práticas. (1) Escreva telos satisfáveis pelos dados reais — campos opcionais devem ser anuláveis. (2) Inclua na especificação a regra de rejeição. (3) Valide programas sintetizados contra invariantes verificáveis sem oráculo (ancoragem na entrada, coerência entre campos) e, quando o custo permitir, contra uma segunda versão independente [4], lembrando que versões geradas pelo mesmo modelo não são independentes [14]. (4) Amostre e audite os programas amortizados: o custo de auditar k programas é pequeno em relação a auditar N registros.

Para além da extração de dados. Nada no mecanismo avaliado é específico de dados. O IDAE é uma instância de um padrão mais geral, que chamamos de *programas descartáveis sob especificação imutável*: um LLM sintetiza um programa por classe de entrada, uma especificação executável fixa decide se ele é aceito, e o programa é reaproveitado até que a classe mude. Nossos resultados sugerem três condições para que o padrão compense. (i) *Especificação verificável e forte*: um validator de formato não alterou nenhuma decisão (E4, E5), enquanto a ancoragem na entrada e o N-version detectaram erros; o benefício cresce à medida que o validator se aproxima de um oráculo. (ii) *Poucas classes e muitos casos por classe* ($k \ll N$): é daí que vem a redução de custo; se cada entrada é única, o padrão degenera em chamar o LLM por caso. (iii) *Erro auditável*: como um programa errado erra de forma sistemática em toda a sua classe, o domínio precisa permitir inspecionar ou amostrar os k programas. Domínios candidatos que satisfazem essas condições incluem a migração de código com teste diferencial, em que o sistema legado serve de oráculo; adaptadores

entre serviços sob testes de contrato; e configuração de infraestrutura sob políticas como código. Onde a especificação é ambígua ou não verificável, o padrão oferece confiança falsa, como o telos estrito do USGS ilustra (E3). Esta avaliação cobre apenas extração de dados; a generalidade do padrão é uma hipótese, não um resultado deste trabalho.

7 Trabalhos Relacionados

Intent-first e síntese de programas. Discussões industriais sobre “intent-first” [23] (citadas como evidência de tendência, não como fundamentação) retomam a programação declarativa [16] e a síntese a partir de especificações [9]. LLMs tornaram a síntese prática [6]; o IDAE usa a síntese como mecanismo de adaptação em tempo de execução, com o programa como artefato descartável.

Sistemas autoadaptativos. O ciclo MAPE-K [13] e os desafios de preservar estabilidade durante a adaptação [7, 20] inspiram a hierarquia de intervenção. O IDAE reage a violações da especificação, e não a métricas operacionais; nossos resultados mostram que adaptar a própria especificação (Fase 2) é arriscado.

Contratos e oráculos. No Design-by-Contract [17], o contrato é passivo; no IDAE, ele guia a síntese. Ambos herdam o problema do oráculo [5]: um contrato verifica propriedades, não valores verdadeiros. Nossa metodologia torna explícita a distância entre as duas coisas.

ETL e data wrangling. Airflow [1] e dbt [8] operam sobre transformações escritas à mão; ferramentas de wrangling [12] inferem transformações por perfil estatístico; abordagens ontológicas alinham schemas conhecidos [21]. O IDAE sintetiza transformações sob demanda para formatos não antecipados.

LLMs em engenharia de software e alucinação. LLMs geram código, reparam programas e sintetizam testes [6, 18, 22]; sistemas multiagente orquestram papéis [10, 19]. A alucinação — saída fluente, porém infiel à fonte — é bem documentada [11]. Mostramos que, em extração, retirar o LLM do caminho crítico reduz um tipo de erro (aritmética por registro) e cria outro (erro sistemático amortizado). A diversidade entre versões geradas pelo mesmo modelo é limitada, como já observado para N-version programming humano [14].

8 Ameaças à Validade

Validade de construto. A corrupção é medida contra um oráculo que escrevemos. As regras de conversão (taxas, fuso UTC, limiares) estão explicitadas na especificação entregue a todas as abordagens, e os testes unitários verificam que toda verdade-terreno satisfaz o validator. Os oráculos do USGS derivam dos campos-fonte, mas o telos corrigido é uma escolha nossa. A tolerância numérica (0,011 em USD; 0,051 em valores de sensores) pode aceitar diferenças de arredondamento.

Validade interna. Os LLMs são não determinísticos e não controlamos a temperatura (usamos a configuração padrão do modelo). Mitigamos isso com repetições (E5, E7) e com o registro de todas as chamadas. Uma primeira execução de E4, E5 e E7 foi interrompida quando identificamos que a rejeição explícita, na versão inicial, não amortizava o programa que rejeita (o que reamostrava o LLM e aumentava a exposição a programas que fabricam dados); os logs

estão preservados no pacote de artefatos, e os resultados reportados são da versão corrigida. A estimativa de tokens por caracteres é aproximada.

Validade externa. Dois domínios sintéticos e um real, com poucos formatos ($k \leq 5$ por domínio). Os resultados com Opus 4.5 e Haiku 4.5 não se generalizam automaticamente para outros modelos; a sensibilidade observada entre os dois sugere que a correção do IDAE depende fortemente do modelo. Não avaliamos modelos abertos locais nesta versão.

Validade de conclusão. Os LLM-baselines rodaram em subamostras de 150 registros por domínio, e o IDAE em fluxos completos; as comparações pareadas usam apenas a interseção. As repetições são 6 por domínio, o mínimo para que um Wilcoxon exato bicaudal possa atingir $p < 0,05$.

9 Considerações Finais

O IDAE torna a intenção o artefato permanente e o programa o artefato descartável. Avaliado com um oráculo de verdade-terreno, o framework cumpre sua promessa de **custo e determinismo**: com um modelo capaz, converteu 560 eventos sísmicos reais sem erros com 2 chamadas, contra 62,7% com 560 chamadas da transformação por registro, e não errou aritmética que o LLM por registro erra.

A promessa **semântica** não se sustenta como propriedade arquitetural. O validator do telos garante apenas o que verifica; a amortização transforma um programa errado em erro sistemático; a reconstrução de validators intermediários corrompeu silenciosamente três quartos das fases em que atuou; e um telos insatisfatório levou o sintetizador a fabricar e a alterar dados reais para satisfazê-lo. Além disso, tratar a rejeição de um registro como falha do programa envenena formatos inteiros. Rejeição explícita amortizada e validação ancorada com N-version reduzem esses efeitos a custo baixo.

Trabalhos futuros: instanciar o padrão em um segundo domínio com especificação forte (migração de código com teste diferencial) para testar a hipótese de que a corrupção silenciosa diminui com a força do telos; revalidação a jusante após a Fase 2; geração de invariantes verificáveis a partir do telos; auditoria amostral de programas amortizados; avaliação com mais formatos ($k > 10$), modelos abertos e domínios com semântica ambígua.

Disponibilidade de Artefatos

O pacote de artefatos contém: implementação do IDAE, da cadeia de intenções e de todas as abordagens; geradores com semente e oráculos; os 560 eventos do USGS; scripts dos experimentos E1–E7; testes unitários; resultados brutos por registro; e o log JSONL de todos os prompts e respostas dos modelos. Os artefatos estão disponíveis em <https://github.com/carloseduardodb/idae>. Este artigo está arquivado no Zenodo sob o DOI 10.5281/zenodo.23050791.

Declaração de Uso de IA Generativa

Além do uso de LLMs como objeto de estudo (Claude Opus 4.5 e Haiku 4.5 como sintetizadores e baselines), foi usado um assistente de IA generativa (Claude) para implementar os scripts de avaliação

e para redigir versões deste texto. O autor revisou o código, os resultados e o texto, e assume responsabilidade integral pelo conteúdo.

Referências

- [1] Apache Airflow. 2023. Apache Airflow Documentation. <https://airflow.apache.org/docs/>
- [2] Andrea Arcuri and Lionel Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. 1–10.
- [3] Karl J. Åström and Björn Wittenmark. 1995. *Adaptive Control*. Addison-Wesley.
- [4] Algirdas Avizienis. 1985. The N-version approach to fault-tolerant software. *IEEE Transactions on Software Engineering* SE-11, 12 (1985), 1491–1501.
- [5] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [7] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, et al. 2009. Software engineering for self-adaptive systems: A research roadmap. In *Software Engineering for Self-Adaptive Systems*. Springer, 1–26.
- [8] dbt Labs. 2024. dbt: Transform data in your warehouse. <https://docs.getdbt.com/>
- [9] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. 2017. Program synthesis. *Foundations and Trends in Programming Languages* 4, 1–2 (2017), 1–119.
- [10] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [11] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.
- [12] Sean Kandel, Andreas Paepcke, Joseph Hellerstein, and Jeffrey Heer. 2011. Wrangler: Interactive visual specification of data transformation scripts. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 3363–3372.
- [13] Jeffrey O. Kephart and David M. Chess. 2003. The vision of autonomic computing. *Computer* 36, 1 (2003), 41–50.
- [14] John C. Knight and Nancy G. Leveson. 1986. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 96–109.
- [15] Barbara Liskov and John Guttag. 2000. *Program Development in Java*. Addison-Wesley.
- [16] John W. Lloyd. 1994. Practical advantages of declarative programming. In *Joint Conference on Declarative Programming (JULP-PRODE’94)*. 18–30.
- [17] Bertrand Meyer. 1997. *Object-Oriented Software Construction* (2 ed.). Prentice Hall.
- [18] Julian Aron Prenner, Hlib Babii, and Romain Robbes. 2022. Can OpenAI’s Codex fix bugs? An evaluation on QuixBugs. In *Proceedings of the Third International Workshop on Automated Program Repair (APR)*. 69–75.
- [19] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, et al. 2024. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [20] Mazeiar Salehie and Ladan Tahvildari. 2009. Self-adaptive software: Landscape and research challenges. *ACM Transactions on Autonomous and Adaptive Systems* 4, 2 (2009), Article 14.
- [21] Paulo Sérgio dos Santos Júnior, João Paulo A. Almeida, and Monalessa Barcellos. 2023. Towards federated ontology-driven data integration in continuous software engineering. In *Proceedings of the XXXVII Brazilian Symposium on Software Engineering (SBES)*. ACM, 432–441.
- [22] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105.
- [23] Ayman Shoukry. 2026. The future of software development: Why coding becomes a specialist sport. *BairesDev Blog* (March 2026). <https://www.bairesdev.com/blog/the-future-of-software-development/>